ISTE-240:- Web & Mobile II

Section: 600

Group: 8

Group Project Final Report

Date: 26/04/2026

Group Members:

Muhammad Usman Habib – 761005924

Yerkebulan Yergeshbay – 759000935

Tanmay Patil – 405007913

Rubric

FRONTEND (20 pts) – CLOS 2, 3 and 5

├── 18-20: Professional + Bootstrap + responsive + toasts + loading states

├── 14-17: Clean + responsive + basic response messages

├── 10-13: Basic CSS + not responsive

└── 0-9: No CSS or broken

REST API (25 pts) – CLOS 4 and 5

├── 22-25: Each student has own controller + All 6 methods

├── 17-21: Each has controller + 4-5 methods

├── 12-16: Only 1-2 controller + 2-3 methods

└── 0-11: Only one controller + Less than 2 methods

JPA (20 pts) – CLO 4

├── 17-20: Each student has own repository + 5+ methods + entity classes

├── 13-16: Each has repository + 3-4 methods

├── 9-12: Only 1-2 repository + 1-2 methods

└── 0-8: One repository for all → ZERO for section

GIT (10 pts) – CLO 1

├── 9-10: Branches + 5+ commits + good messages + PRs

├── 7-8: Branches + 3-4 commits

├── 5-6: One branch + 1-3 commits

└── 0-4: Single commit

DEMO (15 pts) – CLOS 2, 3 and 5

├── 13-15: 5+ pages + all CRUD + no errors

├── 10-12: 4-5 pages + most CRUD

├── 7-9: 3 pages + basic CRUD

└── 0-6: Less than 3 pages

FINAL REPORT (10 pts) – CLO 1

├── 9-10: Complete documentation + API docs + team contributions + screenshots

├── 7-8: Most sections present + basic API docs

├── 5-6: Basic report + missing key sections

└── 0-4: No report or very incomplete

FINAL PRESENTATION (10 pts) – CLO 1

├── 9-10: Clear explanation + live demo works + answers questions confidently

├── 7-8: Good presentation + minor demo issues

├── 5-6: Basic presentation + demo has problems

└── 0-4: Cannot explain code + demo fails

Manifesto

This manifesto shows the contribution of each group member to the ShopKoko project. The team worked collaboratively on planning, implementation, testing, styling, documentation, and presentation preparation. While each member had primary responsibility for particular entities and pages, the final system was developed through shared discussions, design calls, and integration across all parts of the project.

Muhammad Usman Habib

Was primarily responsible for the Product and Category parts of the project. Contributions included creating the Product and Category entities, implementing the corresponding repositories, services, and REST controllers, and building the product-related frontend pages. Created and developed manage_products.html, manage_categories.html, products.html, and product.html, along with their styling and jQuery logic for dynamic interaction with the backend APIs. Also worked on connecting these pages to the Spring Boot backend through AJAX calls and contributed significantly to integrating dynamic CRUD operations on the frontend. Co-authored the final written report together with Yerkebulan.

Yerkebulan Yergeshbay

Was primarily responsible for the Customer and Seller components of the project. Contribution included creating the Customer and Seller entities, implementing the corresponding repositories, services, and REST controllers, and developing the profile and seller management pages. Created profile.html and manage_sellers.html, including the related CSS and jQuery needed to make these pages dynamic and connected to the backend database. Also worked on integrating customer profile display and seller management functionality through the REST API. Co-authored the final written report together with Usman.

Tanmay Patil

Was primarily responsible for the Order portion of the project as well as the project setup and planning. Contributions included creating the Order entity and its corresponding repository, service, and REST controller. Also created the initial landing page structure in index.html, contributed to the website planning, and prepared the presentation materials. Integrated order-related jQuery logic into the profile page so that order information could be displayed dynamically. Also created and maintained the README file and coordinated documentation of project structure, contributions, and setup instructions, including merging all branches to Main.

Shared Team Contributions

All group members contributed to team discussions, design decisions, debugging, and testing of the final integrated system. The web pages were designed collaboratively through group meetings, and major implementation decisions were discussed before integration. The team also worked together to ensure consistent styling, page navigation, database connectivity, and functional integration between frontend and backend components.

Abstract

ShopKoko is a full-stack e-commerce web application designed to provide a user-friendly platform for browsing and managing electronic products. The website allows users to view products, manage orders, and interact with dynamically loaded data through a responsive frontend interface connected to a database. The application is built using a modern architecture combining Spring Boot for backend services, RESTful APIs for data communication, and a MariaDB for database storage. The frontend is developed using HTML, CSS, and jQuery, enabling dynamic interaction with backend APIs. The project demonstrates the implementation of a layered architecture, integrating database operations, business logic, and user interface into one system.

Problem Definition & Solution Context

Many existing e-commerce platforms are either too complex or lack flexibility for customization. There is a need for a simplified system that demonstrates how modern web applications function end-to-end. ShopKoko addresses this by providing a lightweight and simple electronics marketplace that allows users to browse electronic products, view details, and manage orders through a clean and natural interface. The system also includes a practical implementation of RESTful architecture and full-stack development concepts.

Similar Solutions

- **Amazon:** A large-scale global marketplace with a lot of features but high complexity.
- **Noon:** A regional shopping platform offering wide range of products but limited customization.
- **Sharaf DG:** Focused on electronics retail but lacks a proper dynamic user experience.
- **Microless:** A local electronics marketplace with a large variety of products but also very complex to guide through.

These platforms inspired the core concept of ShopKoko while emphasizing simplicity and educational value.

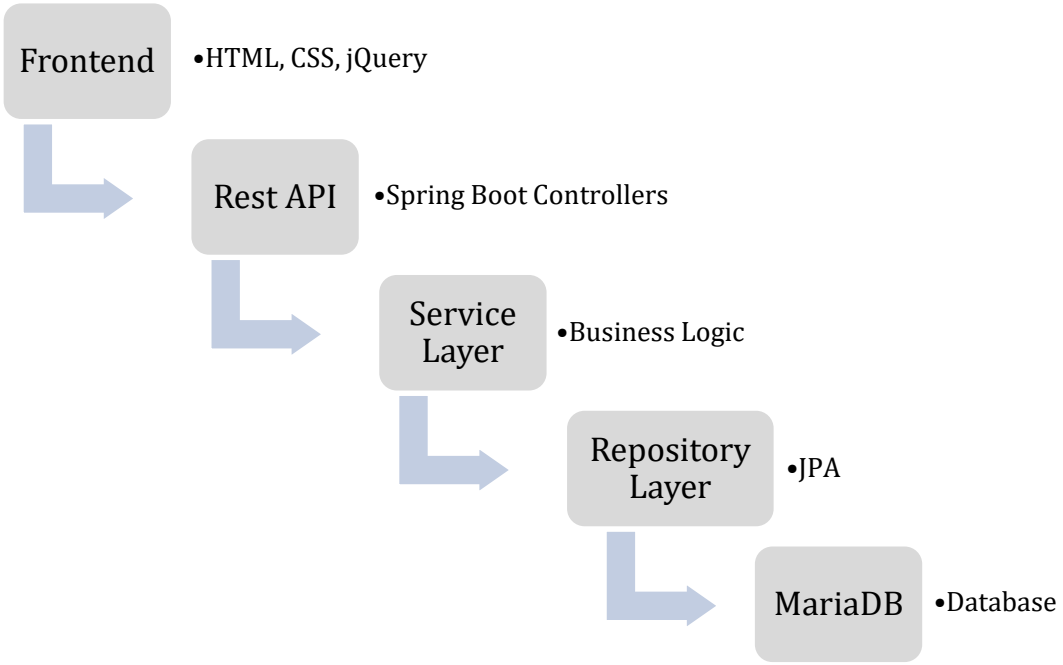
Technologies Used

Layer	Technology Used
Frontend	HTML, CSS, jQuery, Semantic UI
Backend	Java, Spring Boot
Database	MariaDB
API Testing	Postman
Build Tool	Maven

Technical Architecture

ShopKoko follows a layered architecture where each component has a unique role.

Architecture Flow:



The frontend interacts with the backend using AJAX calls to REST endpoints, keeping a clear separation between presentation and data layers.

Key Website Features

- View all available products dynamically
- Search products by name
- View detailed product pages
- Manage customer profiles
- Add, update, and delete records via REST APIs
- Dynamic UI updates using jQuery and AJAX
- Clean and responsive interface design

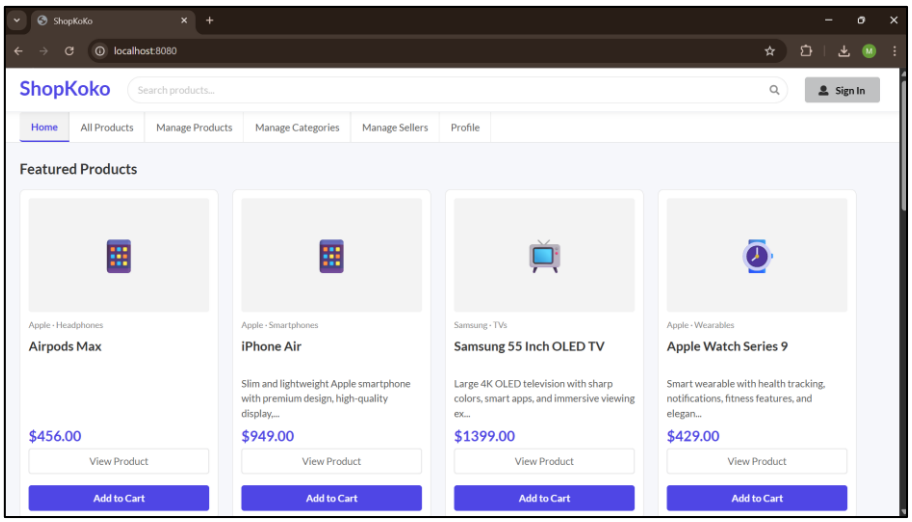
Design Summary

UI/UX Decisions

The website uses a plain and simple interface design with a clean layout to enhance readability and user engagement. Consistent styling and spacing ensure a professional look across all pages.

User Interaction Flow

Users can navigate from the landing page to product catalog, select a product to view details, and access profile or other management pages. Data is dynamically loaded from the backend, ensuring real-time updates.



Database Design & Implementation

The database is implemented using MariaDB and structured using JPA entities.

<u>Entities:</u>

- Product
- Category
- Customer
- Order
- Seller

<u>Relationships:</u>

- Many Products belong to one Category.
- Many Products belong to one Seller.
- Many Orders belong to one Customer.
- Many Orders reference one Product.
- A Category may belong to one parent Category (self-referencing relationship).

The database schema is automatically generated using Hibernate (ddl-auto=update), ensuring consistency between the application and the database structure.

```
MariaDB [iste240]> show tables;
+------------------+
| Tables_in_iste240 |
+------------------+
| categories       |
| customer         |
| orders           |
| products         |
| sellers          |
+------------------+
5 rows in set (0.001 sec)
```

ShopKokoApplication.java | application.properties

```
spring.application.name=shopKoko

spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.datasource.url=jdbc:mariadb://localhost:3306/iste240?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=abc12345

spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.dialect=org.hibernate.dialect.MariaDBDialect
spring.jpa.properties.hibernate.format_sql=true
```

Implementation Details

The backend is developed using Spring Boot following a layered architecture:

- Entities represent database tables
- Repositories handle database operations using JPA
- Services contain business logic and validation
- REST Controllers present API endpoints for CRUD operations

Each entity has a corresponding REST controller supporting:

- GET (all records)
- GET by ID
- GET with search parameters
- POST (create)
- PUT (update)
- DELETE (remove)

The frontend communicates with these endpoints using jQuery AJAX calls, enabling dynamic content rendering.

API Documentation

The ShopKoko backend exposes RESTful API endpoints for each main entity in the system. These endpoints are used by the frontend pages through jQuery AJAX calls to dynamically load, create, update, search, and delete records.

Product API

GET /api/products
Returns all product records.

GET /api/products/{id}
Returns one product by its ID.

GET /api/products/search?name=...
Searches for products by product name.

GET /api/products/available
Returns only products marked as available.

POST /api/products
Creates a new product.

PUT /api/products/{id}
Updates an existing product by ID.

DELETE /api/products/{id}
Deletes a product by ID.

Category API

GET /api/categories
Returns all category records.

GET /api/categories/{id}
Returns one category by its ID.

GET /api/categories/search?name=...
Searches for categories by category name.

GET /api/categories/main
Returns only main categories, meaning categories without a parent category.

POST /api/categories
Creates a new category.

PUT /api/categories/{id}
Updates an existing category by ID.

DELETE /api/categories/{id}
Deletes a category by ID.

Customer API

GET /api/customers
Returns all customer records.

GET /api/customers/{id}
Returns one customer by ID.

GET /api/customers/search?name=...
Searches for customers by customer name.

GET /api/customers/email?value=...
Searches for customers by email.

POST /api/customers
Creates a new customer.

PUT /api/customers/{id}
Updates an existing customer by ID.

DELETE /api/customers/{id}
Deletes a customer by ID.

Order API

GET /api/orders
Returns all order records.

GET /api/orders/{id}
Returns one order by ID.

GET /api/orders/search?status=...
Searches for orders by status.

GET /api/orders/customer/{customerId}
Returns all orders belonging to a specific customer.

POST /api/orders
Creates a new order.

PUT /api/orders/{id}
Updates an existing order by ID.

DELETE /api/orders/{id}
Deletes an order by ID.

Seller API

GET /api/sellers
Returns all seller records.

GET /api/sellers/{id}
Returns one seller by ID.

GET /api/sellers/search?name=...
Searches for sellers by seller name.

POST /api/sellers
Creates a new seller.

PUT /api/sellers/{id}
Updates an existing seller by ID.

DELETE /api/sellers/{id}
Deletes a seller by ID.

Link to GIT repository: https://github.com/mh2923/iste240-team8

Link to recorded demo video:
https://drive.google.com/file/d/1BPzNxPj5KwVUyzaDUWW3y_KydZRIoNns/view?usp=sharing